

DevSecOps Interview Q&A — 120 Questions

DevSecOps — Perguntas e Respostas de Entrevista — 120 Questões

Study set for the Analog Devices Staff DevSecOps pipeline (Casey Galvin screen + whatever technical round follows). Written in English because the real interview will be. Ordered by how likely each topic is to come up, weighted toward the tools practiced hands-on in this crash course (devsecops-crash-course.md, devsecops-scan-report.md) and the exact acronyms named in the job posting (SAST/SCA/SBOM/DAST/SSDLC). AWS and compliance sections are conceptual/definitional where there's a real gap — never claim hands-on experience that isn't there; the honest "I understand X, haven't operated it" answer is in the mix on purpose.

Material de estudo para o processo de Staff DevSecOps da Analog Devices (screen com a Casey Galvin + a rodada técnica que vier depois). Escrito em inglês porque a entrevista real será em inglês. Ordenado pela probabilidade de cada tema aparecer, com peso maior para as ferramentas praticadas na mão neste crash course (devsecops-crash-course.md, devsecops-scan-report.md) e para os acrônimos exatos citados no anúncio da vaga (SAST/SCA/SBOM/DAST/SSDLC). As seções de AWS e compliance são conceituais/definicionais onde existe um gap real — nunca afirmar experiência prática que não existe; a resposta honesta "entendo X, não opereei" está na mistura de propósito.

1. SAST — Static Application Security Testing (15)

1. SAST — Teste Estático de Segurança de Aplicação (15)

1. What is SAST?

1. O que é SAST?

Static Application Security Testing analyzes source code without executing it, looking for known vulnerability patterns by parsing the code into an AST and matching structural patterns against it.

Static Application Security Testing analisa o código-fonte sem executá-lo, procurando padrões conhecidos de vulnerabilidade: converte o código em uma AST (árvore sintática abstrata) e casa padrões estruturais contra ela.

2. How does SAST differ from a plain text-based linter?

2. Qual a diferença entre SAST e um linter baseado em texto?

SAST tools like Semgrep parse code into an abstract syntax tree and match structural patterns, not literal strings — they understand code semantics (variable use, control flow), not just text.

Ferramentas de SAST como o Semgrep convertem o código em uma árvore sintática abstrata e casam padrões estruturais, não strings literais — entendem a semântica do código (uso de variáveis, fluxo de controle), não só o texto.

3. What are SAST's main limitations?

3. Quais são as principais limitações do SAST?

Coverage depends entirely on the loaded ruleset. I proved this myself: Semgrep's `auto` config caught `subprocess.call(..., shell=True)` as command injection but missed the semantically identical `os.system("cmd " + var)` because no rule matched that exact syntactic pattern.

A cobertura depende inteiramente do conjunto de regras carregado. Provei isso na prática: a config `auto` do Semgrep pegou `subprocess.call(..., shell=True)` como command injection, mas deixou passar o `os.system("cmd " + var)` — semanticamente idêntico — porque nenhuma regra casava com aquele padrão sintático exato.

4. How do you close a SAST coverage gap?

4. Como se fecha um gap de cobertura do SAST?

Write a custom rule. I wrote a Semgrep YAML rule matching `os.system("..." + $VAR)` — running both rulesets together went from 3 to 4 findings on the same file. Takes minutes; you don't wait for the vendor's rule registry to catch up.

Escrevendo uma regra própria. Escrevi uma regra YAML de Semgrep casando `os.system("..." + $VAR)` — rodando os dois conjuntos de regras juntos, foram de 3 para 4 achados no mesmo arquivo. Leva minutos; você não fica esperando o registro de regras do fornecedor alcançar o seu caso.

5. Does a clean SAST scan mean the code is secure?

5. Um scan de SAST limpo significa que o código está seguro?

No — it means no loaded rule matched. Zero findings proves absence of *known patterns*, not absence of vulnerabilities.

Não — significa que nenhuma regra carregada casou. Zero achados prova ausência de *padrões conhecidos*, não ausência de vulnerabilidades.

6. What's a common SAST false-negative pattern?

6. Qual é um padrão comum de falso-negativo no SAST?

Semantically equivalent code written differently — the same command-injection bug via two different APIs is a real example I hit personally.

Código semanticamente equivalente escrito de outro jeito — o mesmo bug de command injection por duas APIs diferentes é um exemplo real que vivi.

7. What's a common SAST false-positive source, and how is it handled?

7. Qual é uma fonte comum de falso-positivo no SAST, e como se lida com ela?

Overly broad patterns matching safe code. Teams tune rule specificity or use inline suppressions/allowlists — but suppressions need review, since they can hide real issues introduced later.

Padrões largos demais casando com código seguro. Os times ajustam a especificidade da regra ou usam supressões inline/allowlists — mas supressão precisa de revisão, porque pode esconder um problema real introduzido depois.

8. Is SAST a substitute for code review?

8. SAST substitui code review?

No. It's one layer — it consistently misses semantically-equivalent-but-syntactically-different bugs that a human reviewer might still catch.

Não. É uma camada — deixa passar de forma consistente bugs semanticamente equivalentes mas sintaticamente diferentes, que um revisor humano ainda pegaria.

9. Where does SAST fit in the pipeline?

9. Onde o SAST entra no pipeline?

As early as possible — ideally on every commit/PR, before merge, while the author still has full context.

O mais cedo possível — idealmente a cada commit/PR, antes do merge, enquanto o autor ainda tem todo o contexto na cabeça.

10. Should SAST findings block a PR merge?

10. *Achados de SAST devem bloquear o merge de um PR?*

Depends on maturity. For code under your control, yes for real findings — they're fixable immediately. For legacy code with existing debt, teams often start report-only until the backlog is triaged, or nobody can merge anything.

Depende da maturidade. Para código sob seu controle, sim para achados reais — dá para corrigir na hora. Para código legado com dívida acumulada, os times costumam começar em modo só-reporta até o backlog ser triado, senão ninguém consegue mergear nada.

11. What did you personally build and run with SAST?

11. *O que você construiu e rodou de verdade com SAST?*

Installed and ran Semgrep against a real repo (zero findings, clean code) and against a deliberately vulnerable demo file with command injection, a weak hash, and a hardcoded secret. Diagnosed the coverage gap, wrote and validated a custom rule, confirmed the before/after finding count.

Instalei e rodei o Semgrep contra um repositório real (zero achados, código limpo) e contra um arquivo de demonstração vulnerável de propósito, com command injection, hash fraco e segredo hardcoded. Diagnosticuei o gap de cobertura, escrevi e validei uma regra própria, e confirmei a contagem de achados antes/depois.

12. What format do Semgrep rules use?

12. *Que formato as regras do Semgrep usam?*

YAML, with a `pattern` field written in a DSL that resembles the target language itself, matched structurally against the code's AST.

YAML, com um campo `pattern` escrito numa DSL que se parece com a própria linguagem alvo, casada estruturalmente contra a AST do código.

13. Can SAST find hardcoded secrets?

13. *SAST consegue achar segredos hardcoded?*

Some SAST tools include secret-detection rules — Semgrep flagged a fake Stripe key via a built-in rule — but dedicated secret scanners are usually more complete for that job, especially across git history.

Algumas ferramentas de SAST incluem regras de detecção de segredo — o Semgrep apontou uma chave Stripe falsa por uma regra nativa — mas scanners dedicados a segredo costumam ser mais completos nessa função, principalmente varrendo o histórico do git.

14. What's the risk of relying only on a vendor's default ruleset?

14. *Qual o risco de depender só do conjunto de regras padrão do fornecedor?*

Coverage blind spots specific to your codebase's idioms never get caught. You have to actively audit for what a generic ruleset misses and supplement it.

Pontos cegos de cobertura específicos dos idiomas do seu código nunca são pegos. É preciso auditar ativamente o que um ruleset genérico deixa passar e complementá-lo.

15. How would you introduce SAST into a codebase with zero prior security tooling?

15. *Como você introduziria SAST num código sem nenhuma ferramenta de segurança prévia?*

Start non-blocking/report-only, triage the existing backlog by severity, fix or explicitly accept each finding, then flip it to a blocking gate — otherwise the first blocking run makes every PR unmergeable.

Começando em modo não-bloqueante/só-reporta, triando o backlog existente por severidade, corrigindo ou aceitando explicitamente cada achado, e só então virando a chave para gate bloqueante — senão a primeira rodada bloqueante deixa todo PR impossível de mergear.

2. DAST — Dynamic Application Security Testing (12)

2. DAST — Teste Dinâmico de Segurança de Aplicação (12)

16. What is DAST?

16. O que é DAST?

Dynamic Application Security Testing attacks a running application from the outside, over HTTP, simulating a real attacker. It never reads source code.

Dynamic Application Security Testing ataca uma aplicação em execução por fora, via HTTP, simulando um atacante real. Nunca lê o código-fonte.

17. How does a DAST tool work mechanically?

17. Como uma ferramenta de DAST funciona mecanicamente?

It runs a proxy that first spiders/crawls the app to discover routes, then replays captured requests with variations (missing headers, simple payloads) and audits the responses.

Ela sobe um proxy que primeiro rastreia (spider/crawl) a aplicação para descobrir as rotas, depois repete as requisições capturadas com variações (headers ausentes, payloads simples) e audita as respostas.

18. What can DAST catch that SAST cannot?

18. O que o DAST pega que o SAST não pega?

Runtime/config issues that don't exist as a line of source code. I ran OWASP ZAP against a 12-line FastAPI app with zero logic bugs and it still found 3 missing security headers — a purely runtime characteristic.

Problemas de runtime/configuração que não existem como linha de código-fonte. Rodei o OWASP ZAP contra uma app FastAPI de 12 linhas, sem nenhum bug de lógica, e ele ainda achou 3 headers de segurança ausentes — característica puramente de runtime.

19. What can DAST NOT catch that SAST can?

19. O que o DAST NÃO pega e o SAST pega?

Code paths DAST never exercises because it doesn't know which inputs trigger them — e.g. a command-injection bug behind a specific parameter value it never happened to send.

Caminhos de código que o DAST nunca exercita porque não sabe quais entradas os disparam — por exemplo, um bug de command injection atrás de um valor de parâmetro específico que ele nunca chegou a enviar.

20. What's a "baseline scan" in OWASP ZAP?

20. O que é um "baseline scan" no OWASP ZAP?

A fast, largely passive scan — mostly observes traffic and runs known-check rules rather than actively attacking. Good for CI: quick and non-destructive.

Um scan rápido e majoritariamente passivo — observa o tráfego e roda regras de verificação conhecidas em vez de atacar ativamente. Bom para CI: rápido e não destrutivo.

21. Should DAST block a CI pipeline?

21. O DAST deve bloquear um pipeline de CI?

Often not, at least initially — a baseline finding (a missing header) may need an architecture decision, not a one-line fix. Turning it into a hard gate without triage blocks every merge over things not yet assessed.

Em geral não, pelo menos no início — um achado de baseline (um header ausente) pode exigir uma decisão de arquitetura, não um fix de uma linha. Transformar isso em gate rígido sem triagem bloqueia todo merge por coisas ainda não avaliadas.

22. What did you personally fix using DAST?

22. O que você corrigiu de verdade usando DAST?

Ran ZAP baseline against a running container, got 3 WARN. Added a FastAPI middleware setting `X-Content-Type-Options`, `Cross-Origin-Resource-Policy`, and `Cache-Control` — rebuilt, reran, down to 1 WARN. That last WARN is the expected side effect of `Cache-Control: no-store` — a conscious tradeoff, not a bug.

Rodei o ZAP baseline contra um container em execução e recebi 3 WARN. Adicionei um middleware no FastAPI setando `X-Content-Type-Options`, `Cross-Origin-Resource-Policy` e `Cache-Control` — rebrildei, rodei de novo, caiu para 1 WARN. Esse último WARN é o efeito colateral esperado do `Cache-Control: no-store` — um tradeoff consciente, não um bug.

23. What's an authenticated DAST scan, and why does it matter?

23. O que é um scan DAST autenticado, e por que isso importa?

A scan that logs in first so it can test routes behind auth — most real functionality lives there. I didn't set this up (the demo app has no auth) — a real, acknowledged gap.

Um scan que faz login antes, para conseguir testar as rotas atrás de autenticação — é onde mora a maior parte da funcionalidade real. Não configurei isso (a app de demonstração não tem autenticação) — é um gap real, que eu assumo.

24. DAST vs a manual pentest — what's the difference?

24. DAST versus pentest manual — qual a diferença?

DAST is automated and repeatable but shallow/generic. A pentest is manual, creative, and can chain findings into a real exploit. DAST is baseline hygiene, not a pentest substitute.

DAST é automatizado e repetível, mas raso/genérico. Um pentest é manual, criativo, e consegue encadear achados em um exploit real. DAST é higiene de base, não substituto de pentest.

25. Why run DAST against staging rather than production?

25. Por que rodar DAST em staging e não em produção?

It sends real traffic/attacks — running against production risks disruption, data creation, or tripping real alerts/rate limits.

Porque ele envia tráfego/ataques reais — rodar contra produção arrisca interrupção, criação de dados, ou disparar alertas e rate limits de verdade.

26. What's a typical DAST finding category?

26. Qual é uma categoria típica de achado de DAST?

Missing security headers, verbose error messages leaking stack traces, weak session/cookie flags, outdated JS libraries, basic injection probes.

Headers de segurança ausentes, mensagens de erro verbosas vazando stack trace, flags fracas de sessão/cookie, bibliotecas JS desatualizadas, sondagens básicas de injeção.

27. How do you prioritize DAST findings?

27. Como você prioriza achados de DAST?

By exploitability and exposure — an endpoint leaking version info is lower risk than one accepting unsanitized input that reaches a database or shell.

Por explorabilidade e exposição — um endpoint vazando informação de versão é risco menor que um que aceita entrada não sanitizada chegando a um banco ou a um shell.

3. SCA — Software Composition Analysis (12)

3. SCA — Análise de Composição de Software (12)

28. What is SCA?

28. O que é SCA?

Software Composition Analysis audits third-party dependencies: it extracts name+exact version from a manifest/lockfile and cross-references known CVE databases. It never analyzes the dependency's actual code.

Software Composition Analysis audita dependências de terceiros: extrai nome + versão exata de um manifesto/lockfile e cruza com bases de CVE conhecidas. Nunca analisa o código real da dependência.

29. SCA vs SAST — core difference?

29. SCA versus SAST — qual a diferença essencial?

SAST audits code you wrote; SCA audits code you imported.

SAST audita o código que você escreveu; SCA audita o código que você importou.

30. What did you find running SCA for real?

30. O que você achou rodando SCA de verdade?

Against a `requirements.txt` pinned to old versions (`requests==2.6.0`, `pyyaml==5.1`, `urllib3==1.24.1`), Trivy found 20 real CVEs — 3 CRITICAL, including a PyYAML RCE via `FullLoader` (CVE-2019-20477).

Contra um `requirements.txt` fixado em versões antigas (`requests==2.6.0`, `pyyaml==5.1`, `urllib3==1.24.1`), o Trivy achou 20 CVEs reais — 3 CRITICAL, inclusive um RCE no PyYAML via `FullLoader` (CVE-2019-20477).

31. How did you remediate, and what surprised you?

31. Como você remediou, e o que te surpreendeu?

Pinned each library to its fixed version and rescanned — 20 CVEs down to 0. The surprise: my first fix used a version `range` (`>=`), and Trivy didn't scan at all (`Number of language-specific files: 0`).

Fixei cada biblioteca na versão corrigida e reescaneei — de 20 CVEs para 0. A surpresa: minha primeira correção usou um `range` de versão (`>=`), e o Trivy simplesmente não escaneou (`Number of language-specific files: 0`).

32. Why does a version range break SCA scanning?

32. Por que um range de versão quebra o scan de SCA?

The scanner needs to know the precise installed version to look it up in the vulnerability DB. A range doesn't resolve to one version until installed, so most SCA tools skip unpinned manifests entirely.

O scanner precisa saber a versão exata instalada para consultá-la no banco de vulnerabilidades. Um range só resolve para uma versão no momento da instalação, então a maioria das ferramentas de SCA ignora manifestos não fixados por completo.

33. What's the practical lesson for real CI pipelines?

33. Qual a lição prática para pipelines de CI reais?

Dependencies must be pinned/locked for SCA to mean anything — loose ranges make the security gate blind, not just imprecise.

As dependências precisam estar fixadas/travadas para o SCA significar alguma coisa — ranges soltos deixam o gate de segurança cego, não apenas impreciso.

34. Should SCA findings block a merge?

34. Achados de SCA devem bloquear um merge?

Generally yes for CRITICAL/HIGH in a project under your control — a version bump is low-cost and immediate.

Em geral sim, para CRITICAL/HIGH num projeto sob seu controle — subir de versão é barato e imediato.

35. What tool automates dependency version bumps?

35. *Que ferramenta automatiza a atualização de versão das dependências?*

Dependabot or Renovate. I configured `dependabot.yml` for pip, the Dockerfile base image, and GitHub Actions versions, running weekly.

Dependabot ou Renovate. Configurei o `dependabot.yml` para pip, para a imagem base do Dockerfile e para as versões das GitHub Actions, rodando semanalmente.

36. What's a transitive dependency, and why does it complicate SCA?

36. *O que é uma dependência transitiva, e por que ela complica o SCA?*

A dependency of your dependency, not something you declared directly. A vulnerable transitive package is harder to spot, and sometimes can't be bumped without also bumping (or forking) the direct dependency pinning it.

É a dependência da sua dependência, não algo que você declarou diretamente. Um pacote transitivo vulnerável é mais difícil de enxergar e às vezes não dá para subir sem também subir (ou forkar) a dependência direta que o prende naquela versão.

37. Give an example of a fix that didn't work cleanly.

37. *Dê um exemplo de correção que não funcionou de forma limpa.*

Bumping `fastapi` to pull a patched `starlette` worked (removed 3 HIGH CVEs). But force-upgrading `pip/setuptools/wheel` to fix 2 remaining CVEs just swapped them for two *different* CVEs, net zero gain. I reverted it rather than report a false win.

Subir o `fastapi` para puxar um `starlette` corrigido funcionou (eliminou 3 CVEs HIGH). Mas forçar upgrade de `pip/setuptools/wheel` para fechar 2 CVEs restantes só trocou por dois CVEs *diferentes*, ganho líquido zero. Reverti em vez de reportar uma vitória falsa.

38. How do you verify a fix actually worked?

38. *Como você verifica que uma correção realmente funcionou?*

Always rescan and compare the actual finding list/count — never assume an upgrade reduces risk without re-running the scanner.

Sempre reescaneando e comparando a lista/contagem real de achados — nunca assumir que um upgrade reduz risco sem rodar o scanner de novo.

39. SCA vs SBOM — how do they relate?

39. *SCA e SBOM — como se relacionam?*

SCA does the vulnerability lookup; SBOM is the underlying inventory data that makes lookups fast without a full rescan. Some SCA pipelines consume an existing SBOM instead of re-parsing manifests from scratch.

O SCA faz a consulta de vulnerabilidade; o SBOM é o inventário por baixo que torna essa consulta rápida sem um reescaneamento completo. Alguns pipelines de SCA consomem um SBOM existente em vez de reprocessar os manifestos do zero.

4. SBOM — Software Bill of Materials (10)

4. *SBOM — Lista de Materiais de Software (10)*

40. What is an SBOM?

40. *O que é um SBOM?*

A formal, machine-readable inventory of every component in a software artifact (libraries, versions, sometimes licenses and build-time actions), typically in CycloneDX or SPDX format.

Um inventário formal e legível por máquina de todo componente de um artefato de software (bibliotecas, versões, às vezes licenças e ações de build), tipicamente no formato CycloneDX ou SPDX.

41. Why does SBOM matter when a new CVE drops?

41. Por que o SBOM importa quando sai um CVE novo?

Instead of rescanning your entire fleet, you query already-generated SBOMs to know in minutes which artifacts contain the affected component and version.

Em vez de reescanear o parque inteiro, você consulta os SBOMs já gerados e sabe em minutos quais artefatos contêm o componente e a versão afetados.

42. What did you do to actually test that value?

42. O que você fez para testar esse valor na prática?

Simulated a hypothetical new CVE in `anyio < 4.16.0`, then queried an already-generated `sbom.json` instead of rerunning Trivy — found `anyio 4.15.1` (exposed) in milliseconds.

Simulei um CVE novo hipotético em `anyio < 4.16.0` e consultei um `sbom.json` já gerado em vez de rodar o Trivy de novo — achei `anyio 4.15.1` (exposto) em milissegundos.

43. What tool and format did you use?

43. Que ferramenta e formato você usou?

Syft, output as CycloneDX JSON.

Syft, com saída em CycloneDX JSON.

44. Does scan scope change what's captured?

44. O escopo do scan muda o que é capturado?

Significantly — a directory (source-only) scan of a real repo gave 40 components (including GitHub Actions used in CI); a full container image scan gave 2,872 components, because it captures every file in the image, not just language packages.

Significativamente — um scan de diretório (só código-fonte) de um repositório real deu 40 componentes (incluindo as GitHub Actions usadas no CI); um scan completo da imagem de container deu 2.872 componentes, porque captura todo arquivo da imagem, não só pacotes de linguagem.

45. Is SBOM a security gate?

45. SBOM é um gate de segurança?

No — it's inventory, not a pass/fail check. It's published as a CI artifact for later reference, never blocking anything.

Não — é inventário, não um teste de passa/não passa. É publicado como artefato de CI para referência posterior, nunca bloqueando nada.

46. Why is SBOM a regulatory topic in some industries?

46. Por que SBOM é um tema regulatório em algumas indústrias?

A 2021 US Executive Order pushed federal software supply-chain requirements, and the FDA has required SBOMs in medical device software submissions since 2023 — for a SaMD product line, it can be a submission requirement, not just best practice.

Uma Ordem Executiva americana de 2021 impulsionou requisitos federais de cadeia de suprimentos de software, e o FDA exige SBOM nas submissões de software de dispositivo médico desde 2023 — para uma linha de produto SaMD, pode ser requisito de submissão, não apenas boa prática.

47. CycloneDX vs SPDX?

47. *CycloneDX versus SPDX?*

Both are standard SBOM formats. CycloneDX originated in the OWASP ecosystem with a security/vulnerability focus; SPDX originated in the Linux Foundation with a stronger license-compliance focus. Many tools emit either.

Os dois são formatos padrão de SBOM. O CycloneDX nasceu no ecossistema OWASP com foco em segurança/vulnerabilidade; o SPDX nasceu na Linux Foundation com foco mais forte em conformidade de licença. Muitas ferramentas emitem qualquer um dos dois.

48. What non-obvious thing showed up in your SBOM?

48. *Que coisa não óbvia apareceu no seu SBOM?*

The repo-level SBOM included GitHub Actions used in CI (e.g. `actions/checkout`) as components — SBOM covers supply-chain elements beyond runtime language dependencies.

O SBOM no nível do repositório incluiu como componentes as GitHub Actions usadas no CI (por exemplo `actions/checkout`) — SBOM cobre elementos da cadeia de suprimentos além das dependências de linguagem em runtime.

49. How would you operationalize SBOM at scale?

49. *Como você operacionalizaria SBOM em escala?*

Generate one automatically on every build/release as a pipeline artifact, store them centrally indexed by version, and build simple tooling to query across all of them by component name — that's what turns "SBOM exists" into "SBOM is useful."

Gerando um automaticamente a cada build/release como artefato do pipeline, guardando tudo num repositório central indexado por versão, e criando ferramental simples para consultar todos eles por nome de componente — é isso que transforma "o SBOM existe" em "o SBOM é útil".

5. Container Security / Scanning (10)

5. *Segurança de Container / Escaneamento (10)*

50. Why scan a container image instead of just the app's code?

50. *Por que escanear a imagem do container e não só o código da aplicação?*

The final image includes the OS base layer and its packages — code review never sees that. Measured for real: the full `python:3.11` base image had 654 CRITICAL/HIGH vulnerabilities, almost entirely from the Debian OS, not the app.

A imagem final inclui a camada base do sistema operacional e seus pacotes — code review nunca vê isso. Medido de verdade: a imagem base `python:3.11` completa tinha 654 vulnerabilidades CRITICAL/HIGH, quase todas do Debian, não da aplicação.

51. What did switching base images achieve?

51. *O que a troca de imagem base conseguiu?*

Moving to `python:3.11-slim` dropped it to 59 (a ~91% reduction) without touching a line of app code.

Mudar para `python:3.11-slim` derrubou para 59 (redução de ~91%) sem tocar em uma linha de código da aplicação.

52. How does Trivy scan a container image mechanically?

52. Como o Trivy escaneia uma imagem de container mecanicamente?

It unpacks the image layer by layer, lists installed OS packages (dpkg/apk/rpm metadata) plus language-level packages inside, and cross-references the same CVE database used for filesystem scans.

Ele desempacota a imagem camada por camada, lista os pacotes do sistema operacional instalados (metadados dpkg/apk/rpm) mais os pacotes de linguagem lá dentro, e cruza com o mesmo banco de CVE usado nos scans de sistema de arquivos.

53. What further step did you take beyond `-slim`, and what was the result?

53. Que passo você deu além do `-slim`, e qual foi o resultado?

I first tried a multi-stage build into a `distroless` runtime — it dropped to 49, but still with 2 CRITICAL. Then my CI gate blocked the plain slim image, so I measured the simplest option I had skipped: `apt-get upgrade` in the Dockerfile (the CRITICAL `perl-base` findings had fixes available all along) plus removing `setuptools/wheel`, which the app doesn't use at runtime. That gave 44 findings, 0 CRITICAL, and 0 with `--ignore-unfixed` — better than `distroless` on every number, and it keeps a shell. Lesson: measure the simple option before the sophisticated one.

Primeiro tentei um build multi-stage para um runtime `distroless` — caiu para 49, mas ainda com 2 CRITICAL. Depois o gate do meu CI barrou a imagem slim pura, então medi a opção mais simples que eu tinha pulado: `apt-get upgrade` no Dockerfile (os achados CRITICAL do `perl-base` tinham correção disponível o tempo todo) mais a remoção de `setuptools/wheel`, que a aplicação não usa em runtime. Isso deu 44 achados, 0 CRITICAL, e 0 com `--ignore-unfixed` — melhor que o `distroless` em todos os números, e mantém um shell. Lição: medir a opção simples antes da sofisticada.

54. What's the tradeoff of `distroless`, and did you end up using it?

54. Qual o tradeoff do `distroless`, e você acabou usando?

No shell inside the running container — `docker exec sh` doesn't work, so debugging a live incident depends entirely on logs and external observability. I didn't adopt it: the hardened slim image beat it on the numbers, so paying the debuggability cost bought nothing. `Distroless` makes sense when the residual attack surface actually matters more than operability — I measured instead of assuming.

Sem shell dentro do container em execução — `docker exec sh` não funciona, então depurar um incidente ao vivo depende inteiramente de logs e observabilidade externa. Não adotei: a imagem slim endurecida ganhou dele nos números, então pagar o custo de depurabilidade não compraria nada. `Distroless` faz sentido quando a superfície de ataque residual importa mais que a operabilidade — eu medi em vez de supor.

55. How do you validate a hardened image still works?

55. Como você valida que uma imagem endurecida continua funcionando?

Actually run it and test real behavior — I curled the `/health` endpoint and confirmed a 200 with the expected security headers before trusting the vulnerability-count improvement.

Rodando de verdade e testando o comportamento real — dei `curl` no endpoint `/health` e confirmei 200 com os headers de segurança esperados antes de confiar na melhora da contagem de vulnerabilidades.

56. What do you do with residual vulnerabilities that have no available fix?

56. O que fazer com vulnerabilidades residuais que não têm correção disponível?

Confirm it in the scanner output itself (Trivy reports `Fixed Version: None`) — that's an upstream gap, not an effort gap. It becomes a monitoring item.

Confirmar na própria saída do scanner (o Trivy reporta `Fixed Version: None`) — é um gap do upstream, não falta de esforço. Vira item de monitoramento.

57. How do you catch a new vulnerability in an image that never gets rebuilt?

57. Como pegar uma vulnerabilidade nova numa imagem que nunca é rebuildada?

A scheduled rebuild — I added a weekly cron trigger to CI specifically so the same base-image tag gets rebuilt and rescanned even without a code push, since the tag's *content* can be patched upstream without the tag name changing.

Com rebuild agendado — adicionei um gatilho cron semanal no CI justamente para que a mesma tag de imagem base seja rebuildada e reescaneada mesmo sem push de código, já que o *conteúdo* da tag pode receber patch no upstream sem o nome da tag mudar.

58. What's the blocking policy for container-scan findings?

58. Qual a política de bloqueio para achados de scan de container?

CRITICAL/HIGH block the merge — same logic as SCA: usually fixable by a base-image or dependency change.

CRITICAL/HIGH bloqueiam o merge — mesma lógica do SCA: em geral se resolve com uma troca de imagem base ou de dependência.

59. Give an example of an attempted fix that backfired.

59. Dê um exemplo de tentativa de correção que saiu pela culatra.

Force-upgrading `pip / setuptools / wheel` to close 2 findings swapped them for two different ones from newly-vendored tooling — net zero. Reverted and documented instead of reported as fixed.

Forçar upgrade de `pip / setuptools / wheel` para fechar 2 achados trocou-os por dois outros, vindos de ferramental recém-embutido — saldo zero. Revertido e documentado, em vez de reportado como corrigido.

6. Secrets Management & Scanning (8)

6. Gestão e Escaneamento de Segredos (8)

60. What is secrets scanning?

60. O que é escaneamento de segredos?

Detecting hardcoded credentials — API keys, passwords, tokens — committed into source code or git history.

Detectar credenciais hardcoded — chaves de API, senhas, tokens — commitadas no código-fonte ou no histórico do git.

61. How does Gitleaks work mechanically?

61. Como o Gitleaks funciona mecanicamente?

It walks `git log` commit by commit, not just the current working tree, applying regex and entropy checks against known credential patterns plus custom rules.

Ele percorre o `git log` commit a commit, não só a árvore de trabalho atual, aplicando regex e checagem de entropia contra padrões conhecidos de credencial, mais regras próprias.

62. Why doesn't deleting a leaked key from the file fix the problem?

62. Por que apagar a chave vazada do arquivo não resolve o problema?

The key still exists in the older commit's snapshot. I proved this in a throwaway repo: committed a fake key, "fixed" it in the next commit, and Gitleaks still flagged it, pointing at the earlier commit — `git show` on that commit returned the full key.

A chave continua existindo no snapshot do commit antigo. Provei isso num repositório descartável: comitei uma chave falsa, "corrigi" no commit seguinte, e o Gitleaks continuou apontando o achado no commit anterior — um `git show` naquele commit devolveu a chave inteira.

63. What's the correct remediation for a real leaked credential?

63. Qual a remediação correta para uma credencial realmente vazada?

Rotate it immediately — treat the old value as permanently compromised regardless of what happens to the repo. Rewriting history is cleanup, not the fix, and it breaks existing clones.

Rotacionar imediatamente — tratar o valor antigo como permanentemente comprometido, não importa o que aconteça com o repositório. Reescrever o histórico é limpeza, não a correção, e quebra os clones existentes.

64. Where should secrets scanning run?

64. Onde o escaneamento de segredos deve rodar?

Two points ideally: a local pre-commit hook (catches it before a commit exists) and CI (a safety net if the hook wasn't installed or was bypassed).

Em dois pontos, idealmente: um hook de pre-commit local (pega antes de o commit existir) e o CI (rede de segurança se o hook não foi instalado ou foi contornado).

65. What did you build to enforce this locally?

65. O que você construiu para impor isso localmente?

A `.pre-commit-config.yaml` running the official Gitleaks hook on every commit. Tested both directions — blocked a real random AWS-format key (exit code 1, commit rejected) and let through `AKIAIOSFODNN7EXAMPLE`.

Um `.pre-commit-config.yaml` rodando o hook oficial do Gitleaks a cada commit. Testei nos dois sentidos — bloqueou uma chave real em formato AWS gerada aleatoriamente (exit code 1, commit rejeitado) e deixou passar `AKIAIOSFODNN7EXAMPLE`.

66. Why did that specific key pass, and what's the lesson?

66. Por que aquela chave específica passou, e qual é a lição?

`AKIAIOSFODNN7EXAMPLE` is AWS's own documentation placeholder, on Gitleaks' default allowlist so tutorials don't create noise. Lesson: every scanner ships default allowlists — never validate a hook using a vendor's canonical "example" credential, or you'll wrongly conclude it doesn't work.

`AKIAIOSFODNN7EXAMPLE` é o placeholder da própria documentação da AWS, e está na allowlist padrão do Gitleaks para que tutoriais não gerem ruído. Lição: todo scanner vem com allowlist padrão — nunca valide um hook usando a credencial de "exemplo" canônica de um fornecedor, ou você vai concluir erradamente que ele não funciona.

67. Should secrets ever be committed "for a demo"?

67. Alguma vez vale commitar segredo "só para uma demonstração"?

No — even documentation describing a fake secret can trip real scanners (GitHub's own push protection blocked a commit of mine over a fake Stripe key pasted into a markdown doc). Redact or describe it; never paste anything matching a real credential's shape.

Não — até documentação descrevendo um segredo falso pode disparar scanners reais (o push protection do próprio GitHub barrou um commit meu por causa de uma chave Stripe falsa colada num documento markdown). Redija ou descreva; nunca cole nada com o formato de uma credencial real.

7. SSDLC & CI/CD Security Gates (10)

7. SSDLC e Gates de Segurança no CI/CD (10)

68. What is SSDLC?

68. O que é SSDLC?

Secure Software Development Lifecycle — not a tool, a *process*: security built into every phase (design, code, build, deploy, operate), not a single scan bolted on before release.

Secure Software Development Lifecycle (ciclo de vida seguro de desenvolvimento de software) — não é ferramenta, é *processo*: segurança embutida em cada fase (design, código, build, deploy, operação), não um único scan pregado antes do release.

69. How do the individual tools map onto SSDLC phases?

69. Como cada ferramenta se encaixa nas fases do SSDLC?

SAST and secrets scanning run at code/commit time; SCA and container scanning run at build time; SBOM is generated at build/release time; DAST runs against a deployed environment.

SAST e escaneamento de segredos rodam no momento do código/commit; SCA e scan de container rodam no build; o SBOM é gerado no build/release; o DAST roda contra um ambiente já implantado.

70. Blocking vs non-blocking gate — what's the difference?

70. Gate bloqueante versus não bloqueante — qual a diferença?

Blocking gates fail the pipeline and stop the merge/deploy; non-blocking gates report findings without stopping anything.

Gates bloqueantes derrubam o pipeline e impedem o merge/deploy; gates não bloqueantes reportam os achados sem impedir nada.

71. How do you decide which gates block?

71. Como você decide quais gates bloqueiam?

By how directly fixable the finding is and how much control you have. In my pipeline: SAST, secrets, and SCA (CRITICAL/HIGH) block — fixable in code/deps you control. Container scan blocks on CRITICAL/HIGH. SBOM never blocks. DAST only reports, since a baseline finding can need an architecture decision.

Pelo quanto o achado é diretamente corrigível e quanto controle você tem. No meu pipeline: SAST, segredos e SCA (CRITICAL/HIGH) bloqueiam — são corrigíveis em código ou dependências sob seu controle. O scan de container bloqueia em CRITICAL/HIGH. O SBOM nunca bloqueia. O DAST só reporta, já que um achado de baseline pode exigir uma decisão de arquitetura.

72. What happens if every gate is blocking on day one in a legacy codebase?

72. O que acontece se todo gate for bloqueante no primeiro dia num código legado?

Nobody can merge anything — the existing backlog has to be triaged (fixed, or explicitly accepted/suppressed) before the check becomes a hard gate.

Ninguém consegue mergear nada — o backlog existente precisa ser triado (corrigido, ou explicitamente aceito/suprimido) antes de a verificação virar gate rígido.

73. How do you keep dependencies and base images from silently rotting?

73. Como impedir que dependências e imagens base apodreçam em silêncio?

Automate it — Dependabot for version bumps, plus a scheduled CI rebuild to catch security patches landing on the same tag without a version bump.

Automatizando — Dependabot para subir versões, mais um rebuild agendado no CI para pegar patches de segurança que chegam na mesma tag sem mudança de versão.

74. What's "shift-left" security?

74. O que é segurança "shift-left"?

Moving checks as early as possible in the pipeline — e.g. SAST/secrets scanning pre-commit rather than only at deploy time, so the person with full context fixes it immediately.

Mover as verificações o mais cedo possível no pipeline — por exemplo, SAST e escaneamento de segredos no pre-commit em vez de só no deploy, para que quem está com todo o contexto corrija na hora.

75. What proves a pipeline actually works end-to-end?

75. O que prova que um pipeline funciona de ponta a ponta?

Running it against a real PR and watching the jobs execute. I pushed a trivial change and opened a PR, and the real runs found four problems the paper version hid: (1) I'd guessed a nonexistent action tag (`trivy-action@0.28.0` — the repo uses `v0.36.0`); (2) the container gate blocked the very findings I'd documented as accepted, forcing a real policy decision (block only what has a fix available, via `ignore-unfixed`); (3) the ZAP action failed with "Resource not accessible by integration" because the default `GITHUB_TOKEN` can't create issues; (4) after that, its artifact upload failed because the pinned action version used a retired GitHub artifacts API — fixed by bumping it. The ZAP scan itself matched my local result exactly every time (66 PASS, 1 WARN). After those fixes all six jobs went green. None of that shows up until the pipeline actually runs.

Rodar contra um PR real e ver os jobs executando. Subi uma mudança trivial, abri um PR, e as execuções reais acharam quatro problemas que a versão no papel escondia: (1) eu tinha chutado uma tag de action inexistente (`trivy-action@0.28.0` — o repositório usa `v0.36.0`); (2) o gate de container barrou exatamente os achados que eu tinha documentado como aceitos, forçando uma decisão real de política (bloquear só o que tem correção disponível, via `ignore-unfixed`); (3) a action do ZAP falhou com "Resource not accessible by integration" porque o `GITHUB_TOKEN` padrão não pode criar issues; (4) depois disso, o upload do artefato dela falhou porque a versão fixada usava uma API de artefatos aposentada do GitHub — resolvido subindo a versão. O scan do ZAP em si bateu exatamente com o resultado local todas as vezes (66 PASS, 1 WARN). Depois dessas correções os seis jobs ficaram verdes. Nada disso aparece enquanto o pipeline não roda de verdade.

76. How do you reduce alert fatigue across multiple security tools?

76. Como reduzir a fadiga de alertas com várias ferramentas de segurança?

Tune each ruleset to actual risk tolerance, route only actionable findings to block merges, keep exploratory findings (like DAST baseline warnings) as reports until triaged.

Ajustando cada ruleset à tolerância real de risco, deixando só os achados acionáveis bloquearem merges, e mantendo os achados exploratórios (como os avisos de baseline do DAST) como relatório até serem triados.

77. What's a realistic first-90-days plan to add SSDLC gates to an ungated pipeline?

77. Qual um plano realista de primeiros 90 dias para adicionar gates de SSDLC a um pipeline sem nenhum?

Land each scanner report-only first, triage the initial backlog by severity, fix or accept each item, then flip the highest-confidence/lowest-noise checks (secrets, SCA CRITICAL) to blocking first, and expand from there.

Colocar cada scanner primeiro em modo só-relatório, triar o backlog inicial por severidade, corrigir ou aceitar cada item, depois virar para bloqueante primeiro as verificações de maior confiança e menor ruído (segredos, SCA CRITICAL), e expandir a partir daí.

8. AWS Security (18)

8. Segurança em AWS (18)

78. What is IAM?

78. O que é IAM?

AWS's Identity and Access Management service — controls who (users, roles, services) can do what to which resources, via policies.

O serviço de Identity and Access Management da AWS — controla quem (usuários, roles, serviços) pode fazer o quê em quais recursos, por meio de policies.

79. IAM user vs IAM role?

79. Usuário IAM versus role IAM?

A user is a long-lived identity with its own credentials; a role is assumed temporarily with short-lived credentials. Roles are generally preferred for automation — no long-lived secret to leak.

Um usuário é uma identidade de longa duração com credenciais próprias; uma role é assumida temporariamente, com credenciais de curta duração. Roles são geralmente preferíveis para automação — não há segredo de longa duração para vaziar.

80. What's least privilege in an IAM context?

80. O que é menor privilégio no contexto de IAM?

Granting only the specific permissions a user/role/service needs — nothing broader "just in case."

Conceder apenas as permissões específicas de que um usuário/role/serviço precisa — nada mais amplo "por via das dúvidas".

81. What is KMS?

81. O que é KMS?

AWS Key Management Service — a managed service for creating and controlling encryption keys used to protect data at rest (and sometimes in transit).

AWS Key Management Service — serviço gerenciado para criar e controlar chaves de criptografia usadas para proteger dados em repouso (e, às vezes, em trânsito).

82. How does KMS differ from encrypting something yourself?

82. Qual a diferença entre o KMS e criptografar por conta própria?

KMS centralizes key lifecycle (rotation, access policy, audit logging via CloudTrail) instead of each app/service managing its own raw key material.

O KMS centraliza o ciclo de vida da chave (rotação, política de acesso, log de auditoria via CloudTrail) em vez de cada aplicação/serviço gerenciar o próprio material bruto de chave.

83. What is Cognito?

83. O que é o Cognito?

AWS's managed user identity/authentication service — sign-up, sign-in, and access control for apps, including federation with external identity providers.

O serviço gerenciado de identidade/autenticação de usuários da AWS — cadastro, login e controle de acesso para aplicações, incluindo federação com provedores de identidade externos.

84. ECS vs EKS?

84. ECS versus EKS?

Both run containers on AWS. ECS is AWS's own proprietary orchestrator; EKS is AWS's managed Kubernetes offering — same problem category, different control plane.

Os dois rodam containers na AWS. O ECS é o orquestrador proprietário da própria AWS; o EKS é a oferta de Kubernetes gerenciado da AWS — mesma categoria de problema, control plane diferente.

85. If you've only run self-managed Kubernetes, how do you speak to EKS?

85. Se você só operou Kubernetes self-managed, como falar sobre EKS?

Honestly: no hands-on with AWS's managed control plane, but real experience administering a Kubernetes control plane end-to-end (upgrades, node failure, storage, secrets, networking) — arguably more operational surface than only consuming a managed service.

Com honestidade: sem prática no control plane gerenciado da AWS, mas com experiência real administrando um control plane de Kubernetes de ponta a ponta (upgrades, falha de nó, armazenamento, segredos, rede) — discutivelmente mais superfície operacional do que apenas consumir um serviço gerenciado.

86. What is AWS Lambda?

86. O que é o AWS Lambda?

A serverless compute service — deploy a function, AWS runs it on demand without you managing the underlying server.

Um serviço de computação serverless — você publica uma função e a AWS a executa sob demanda, sem você gerenciar o servidor por baixo.

87. What security concerns are specific to Lambda?

87. Que preocupações de segurança são específicas do Lambda?

The execution role's permissions (over-privileged functions are common), dependency vulnerabilities bundled into the deployment artifact, and environment variable handling for secrets.

As permissões da execution role (funções com privilégio excessivo são comuns), vulnerabilidades de dependência embutidas no artefato de deploy, e o tratamento de variáveis de ambiente para segredos.

88. What is RDS?

88. O que é o RDS?

AWS's managed relational database service — AWS handles patching, backups, and failover infrastructure.

O serviço gerenciado de banco de dados relacional da AWS — a AWS cuida de patching, backups e infraestrutura de failover.

89. Security-relevant difference between RDS and a self-hosted database?

89. Diferença relevante de segurança entre RDS e um banco self-hosted?

RDS handles OS/engine patching and backup infrastructure, but you're still responsible for network exposure (security groups/VPC), access control, and encryption configuration.

O RDS cuida do patching de sistema operacional/engine e da infraestrutura de backup, mas você continua responsável pela exposição de rede (security groups/VPC), pelo controle de acesso e pela configuração de criptografia.

90. What is S3, and what's a classic S3 security mistake?

90. O que é o S3, e qual o erro clássico de segurança com ele?

AWS's object storage service. Classic mistake: an accidentally public bucket (misconfigured policy/ACL) exposing sensitive data — a recurring root cause in real breach postmortems.

O serviço de armazenamento de objetos da AWS. Erro clássico: um bucket acidentalmente público (policy/ACL mal configurada) expondo dado sensível — causa raiz recorrente em postmortems de vazamentos reais.

91. What is SQS?

91. O que é o SQS?

AWS's managed message queue service, used to decouple services — a producer writes a message, a consumer processes it later, absorbing load spikes.

O serviço gerenciado de fila de mensagens da AWS, usado para desacoplar serviços — um produtor escreve a mensagem, um consumidor processa depois, absorvendo picos de carga.

92. What is CloudFormation?

92. O que é o CloudFormation?

AWS's native infrastructure-as-code service — declare resources in a template, AWS provisions/updates them to match.

O serviço nativo de infraestrutura como código da AWS — você declara os recursos num template e a AWS provisiona/atualiza para corresponder.

93. CloudFormation vs Terraform?

93. CloudFormation versus Terraform?

Same category (declarative IaC). CloudFormation is AWS-native, only manages AWS; Terraform is multi-cloud/provider-agnostic via its provider ecosystem.

Mesma categoria (IaC declarativa). O CloudFormation é nativo da AWS e só gerencia AWS; o Terraform é multi-cloud/agnóstico de provedor, via seu ecossistema de providers.

94. What AWS service continuously checks for account misconfigurations?

94. Que serviço da AWS verifica continuamente configurações erradas na conta?

AWS Config (compliance rules) or Security Hub (aggregated findings) for misconfig; GuardDuty for threat detection specifically.

AWS Config (regras de conformidade) ou Security Hub (achados agregados) para configuração errada; GuardDuty especificamente para detecção de ameaça.

95. If production runs on Lightsail today, how do you talk about evolving to a standard AWS security posture?

95. Se a produção roda em Lightsail hoje, como falar sobre evoluir para uma postura de segurança AWS padrão?

Frame it as future evolution, not existing experience: "today it runs on Lightsail; at greater scale I'd evaluate VPC-isolated EC2/ECS with proper IAM roles, KMS-backed encryption, and CloudWatch monitoring" — shows the destination without inventing claims you don't have.

Enquadrando como evolução futura, não como experiência existente: "hoje roda em Lightsail; em escala maior eu avaliaria EC2/ECS isolados em VPC, com roles IAM adequadas, criptografia apoiada em KMS e monitoramento via CloudWatch" — mostra o destino sem inventar claims que você não tem.

9. Compliance & Regulated Software (15)

9. Conformidade e Software Regulado (15)

96. What is HIPAA, and why does it matter to a DevSecOps engineer?

96. O que é HIPAA, e por que importa para um engenheiro de DevSecOps?

The US law governing protection of health data (PHI). It constrains how systems log, store, transmit, and grant access to that data — engineering decisions have direct compliance consequences.

A lei americana que rege a proteção de dados de saúde (PHI). Ela restringe como os sistemas logam, armazenam, transmitem e concedem acesso a esse dado — decisões de engenharia têm consequência direta de conformidade.

97. What is SaMD?

97. O que é SaMD?

Software as a Medical Device — an FDA regulatory category for software that itself functions as a medical device, with its own lifecycle and risk-management requirements.

Software as a Medical Device (software como dispositivo médico) — categoria regulatória do FDA para software que por si só funciona como dispositivo médico, com requisitos próprios de ciclo de vida e gestão de risco.

98. How does SaMD differ from software embedded in a regulated physical device?

98. Qual a diferença entre SaMD e software embarcado num dispositivo físico regulado?

SaMD IS the device from a regulatory standpoint; embedded software is regulated as part of the hardware's overall approval — different regulatory pathway.

Do ponto de vista regulatório, o SaMD É o dispositivo; software embarcado é regulado como parte da aprovação geral do hardware — caminho regulatório diferente.

99. What is IEC 62304?

99. O que é a IEC 62304?

An international standard for medical device software lifecycle processes — defines required activities/documentation across the development lifecycle.

Norma internacional para os processos de ciclo de vida de software de dispositivo médico — define as atividades e a documentação exigidas ao longo do desenvolvimento.

100. What is ISO 14971?

100. O que é a ISO 14971?

An international standard for risk management applied to medical devices — a structured process for identifying, evaluating, and controlling risk across the product's life.

Norma internacional de gestão de risco aplicada a dispositivos médicos — um processo estruturado para identificar, avaliar e controlar risco ao longo da vida do produto.

101. How do IEC 62304 and ISO 14971 relate?

101. Como a IEC 62304 e a ISO 14971 se relacionam?

IEC 62304 governs the software lifecycle process; ISO 14971 governs the risk-management process feeding into decisions made throughout that lifecycle — complementary, not overlapping.

A IEC 62304 rege o processo de ciclo de vida do software; a ISO 14971 rege o processo de gestão de risco que alimenta as decisões tomadas ao longo desse ciclo — complementares, não sobrepostas.

102. What is ISO 27001?

102. O que é a ISO 27001?

An international standard/certification for an organization's Information Security Management System — governance/process-level, not a per-product audit.

Norma/certificação internacional para o Sistema de Gestão de Segurança da Informação de uma organização — nível de governança/processo, não auditoria por produto.

103. What is SOC 2?

103. O que é SOC 2?

A US audit framework/report attesting a service organization's controls meet defined trust criteria — commonly requested by enterprise customers during vendor due diligence.

Framework/relatório de auditoria americano atestando que os controles de uma organização prestadora de serviço atendem a critérios de confiança definidos — comumente exigido por clientes corporativos na due diligence de fornecedor.

104. ISO 27001 vs SOC 2 — practical difference?

104. ISO 27001 versus SOC 2 — diferença prática?

ISO 27001 is a certification with an ongoing management system; SOC 2 is an audit report (Type I or Type II) usually requested contractually by US enterprise customers. Different audience, similar underlying intent.

A ISO 27001 é uma certificação com um sistema de gestão contínuo; o SOC 2 é um relatório de auditoria (Tipo I ou Tipo II) normalmente exigido em contrato por clientes corporativos americanos. Público diferente, intenção de fundo parecida.

105. Why does SBOM matter specifically for a SaMD product?

105. Por que o SBOM importa especificamente para um produto SaMD?

The FDA has required SBOMs in medical device software submissions since 2023 — for that product line it's a submission requirement, not just best practice.

O FDA exige SBOM nas submissões de software de dispositivo médico desde 2023 — para essa linha de produto é requisito de submissão, não apenas boa prática.

106. What experience is relevant to regulated health data, even without SaMD-specific work?

106. Que experiência é relevante para dado de saúde regulado, mesmo sem trabalho específico em SaMD?

Two years supporting a lab operating under HIPAA and equipment under clinical regulation — real exposure to the operational discipline regulated environments require, even without the SaMD/IEC 62304 process itself.

Dois anos dando suporte a um laboratório operando sob HIPAA e a equipamentos sob regulação clínica — exposição real à disciplina operacional que ambientes regulados exigem, mesmo sem o processo de SaMD/IEC 62304 em si.

107. How would you approach a codebase under IEC 62304/FDA scope if you've never worked in that process before?

107. Como você abordaria um código sob escopo IEC 62304/FDA se nunca trabalhou nesse processo?

Ask explicitly what documentation/traceability the process requires before changing anything — the paper trail (requirements → design → test → risk assessment) is often as load-bearing as the code, and skipping it can invalidate a submission.

Perguntando explicitamente que documentação/rastreabilidade o processo exige antes de mudar qualquer coisa — a trilha documental (requisito → design → teste → avaliação de risco) muitas vezes sustenta tanto quanto o código, e pular isso pode invalidar uma submissão.

108. What's a "risk-based" approach in ISO 14971 terms?

108. O que é uma abordagem "baseada em risco" nos termos da ISO 14971?

Prioritizing controls/mitigation effort by the severity and likelihood of potential harm, documented and revisited across the lifecycle — not a one-time checklist.

Priorizar controles e esforço de mitigação pela severidade e probabilidade do dano potencial, documentando e revisitando ao longo do ciclo de vida — não é um checklist de uma vez só.

109. Why might a company be conservative about full CI/CD automation in a regulated product line?

109. Por que uma empresa seria conservadora com automação total de CI/CD numa linha de produto regulada?

Changes may need to trace to a formal risk assessment and validation record before release — full automation without a compliance gate can undermine the audit trail regulators expect.

As mudanças podem precisar ser rastreadas até uma avaliação de risco formal e um registro de validação antes do release — automação total sem um gate de conformidade pode corroer a trilha de auditoria que os reguladores esperam.

110. What is OWASP, in one sentence?

110. O que é a OWASP, em uma frase?

A nonprofit/community publishing widely-referenced application security resources — most famously the OWASP Top 10.

Uma comunidade/organização sem fins lucrativos que publica materiais de segurança de aplicação amplamente referenciados — o mais famoso deles, o OWASP Top 10.

10. OWASP Top 10 & AppSec Fundamentals (8)

10. OWASP Top 10 e Fundamentos de AppSec (8)

111. What's in the OWASP Top 10, broadly?

111. O que tem no OWASP Top 10, em linhas gerais?

Categories like broken access control, cryptographic failures, injection, insecure design, security misconfiguration, vulnerable/outdated components, authentication failures, integrity failures, insufficient logging, and SSRF — exact ranking shifts between revisions, categories recur.

Categorias como controle de acesso quebrado, falhas criptográficas, injeção, design inseguro, configuração incorreta de segurança, componentes vulneráveis/desatualizados, falhas de autenticação, falhas de integridade, logging insuficiente e SSRF — a ordem exata muda entre as revisões, as categorias se repetem.

112. What is command injection, concretely?

112. O que é command injection, concretamente?

Untrusted input reaching a shell command without escaping/parameterization — e.g. `os.system("tar -czf backup.tar.gz " + filename)`, where an attacker-controlled `filename` injects shell syntax.

Entrada não confiável chegando a um comando de shell sem escape/parametrização — por exemplo `os.system("tar -czf backup.tar.gz " + filename)`, em que um `filename` controlado pelo atacante injeta sintaxe de shell.

113. Why is `shell=True` risky in `subprocess` calls?

113. Por que `shell=True` é arriscado em chamadas de `subprocess`?

It spawns the command through a shell, so shell metacharacters in the input are interpreted, not passed as a literal argument. Fix: `shell=False` with arguments as a list.

Porque o comando é executado através de um shell, então metacaracteres de shell na entrada são interpretados em vez de passados como argumento literal. Correção: `shell=False` com os argumentos em lista.

114. Why is MD5 unsuitable for password hashing?

114. Por que MD5 não serve para hash de senha?

It's fast and unsalted by design, making brute-force/rainbow-table attacks cheap. Fix: a slow, salted algorithm like bcrypt, scrypt, or Argon2.

Porque é rápido e, por construção, sem salt, o que torna barato o ataque de força bruta ou rainbow table. Correção: um algoritmo lento e com salt, como bcrypt, scrypt ou Argon2.

115. What's a security misconfiguration, concretely, from your own testing?

115. O que é uma configuração incorreta de segurança, concretamente, nos seus testes?

A missing `X-Content-Type-Options` header, or unnecessarily cacheable content on a sensitive endpoint — small runtime defaults that add up to real exposure.

Um header `X-Content-Type-Options` ausente, ou conteúdo desnecessariamente cacheável num endpoint sensível — pequenos padrões de runtime que somados viram exposição real.

116. Authentication vs authorization?

116. Autenticação versus autorização?

Authentication verifies who you are; authorization determines what you're allowed to do once identified. A broken-access-control bug is an authorization failure even with correct authentication.

Autenticação verifica quem você é; autorização determina o que você pode fazer depois de identificado. Um bug de controle de acesso quebrado é falha de autorização mesmo com a autenticação correta.

117. What's a vulnerable/outdated components issue, in your own words?

117. O que é um problema de componente vulnerável/desatualizado, nas suas palavras?

Exactly what SCA catches — a dependency with a known CVE the team hasn't upgraded, sometimes for years (e.g. `pyyaml==5.1`, RCE-vulnerable since 2019).

Exatamente o que o SCA pega — uma dependência com CVE conhecido que o time não atualizou, às vezes por anos (por exemplo `pyyaml==5.1`, vulnerável a RCE desde 2019).

118. Why does defense in depth matter across all these tools?

118. Por que defesa em profundidade importa entre todas essas ferramentas?

No single tool covers everything — SAST misses semantically-equivalent bugs, DAST misses unreachable code paths, SCA doesn't look at your own code, secrets scanning doesn't catch logic bugs. Overlapping layers catch what any one layer misses.

Nenhuma ferramenta sozinha cobre tudo — o SAST deixa passar bugs semanticamente equivalentes, o DAST não alcança caminhos de código não exercitados, o SCA não olha o seu próprio código, o escaneamento de segredos não pega bug de lógica. Camadas sobrepostas pegam o que cada camada isolada deixa passar.

11. Wrap-up (2)

11. Fechamento (2)

119. What's your biggest DevSecOps gap right now?

119. Qual é o seu maior gap em DevSecOps hoje?

I hadn't worked hands-on with SAST/DAST/SCA/SBOM tooling formally before this — closed that by installing and running each tool against real code, deliberately provoking failures, and fixing what I could reverify. Still open: rotating a credential after an actual production leak, and an authenticated DAST scan — both need conditions I didn't have available to practice safely.

Eu não tinha trabalhado formalmente, na prática, com ferramental de SAST/DAST/SCA/SBOM antes disto — fechei esse gap instalando e rodando cada ferramenta contra código real, provocando falhas de propósito e corrigindo o que eu conseguia reverificar. Continua em aberto: rotacionar uma credencial depois de um vazamento real em produção, e um scan DAST autenticado — os dois exigem condições que eu não tinha disponíveis para praticar com segurança.

120. How do you stay current on emerging security tooling?

120. Como você se mantém atualizado sobre ferramental de segurança emergente?

Hands-on, not passive reading. When I hit an unfamiliar tool, I install it, run it against something real (or a deliberately vulnerable target), break it on purpose, and document what actually happened — not what the docs claim should happen.

Na prática, não por leitura passiva. Quando encontro uma ferramenta desconhecida, eu instalo, rodo contra algo real (ou contra um alvo vulnerável de propósito), quebro de propósito, e documento o que aconteceu de fato — não o que a documentação diz que deveria acontecer.